
Applied AI-Assisted Cisco Network Troubleshooting with Human-in-the-Loop Review

Project Type
Applied AI Network Engineering
Domain
Cisco Packet Tracer & Lab Environments
Architecture
AI Diagnosis · Rule Checking · Human Review
Stack
React · TypeScript · Convex · Gemini · Python
Date
August 2026

NetSage AI

NetSage AI — Applied AI + Network Troubleshooting

with Human Review

Project Title: NetSage AI — Applied AI-Assisted Cisco Network Troubleshooting with Human-in-the-Loop Review

2. Problem Statement

Network troubleshooting in Cisco Packet Tracer and lab environments is a time-intensive, knowledge-dependent process. Students and junior engineers often struggle to diagnose faults systematically because they lack the experience to narrow down root causes from show-command output, topology notes, and symptom descriptions. Common issues such as VLAN misconfigurations, OSPF adjacency failures, ACL errors, and DHCP pool exhaustion follow repeatable diagnostic patterns, yet these patterns are not easily accessible to learners. Existing tools provide either static rule-based checking (which cannot reason about context) or general-purpose AI (which lacks networking domain grounding and cannot provide evidence-based, auditable diagnoses). There is a need for a purpose-built system that combines AI-generated diagnostics with deterministic rule checking, a structured case library, and mandatory human review — ensuring that every AI recommendation is validated before any network configuration is changed.

3. Project Objective

The objective of NetSage AI is to build an AI-assisted network troubleshooting system that:

1. Accepts user-described network symptoms, Packet Tracer topology notes, and Cisco IOS show-command output.
1. Produces a structured, evidence-based diagnosis using a large language model (Google Gemini).
1. Runs an independent, deterministic Python rule checker to validate network configurations against six common fault categories.
2. Provides a curated dataset of 30 realistic troubleshooting cases spanning 8 networking categories.
1. Enforces a mandatory human review workflow where every AI diagnosis must be Accepted, Edited, or Rejected before it is considered reliable.
1. Maintains a transparent Responsible AI audit log recording every AI decision, confidence score, and human override.
1. Presents all metrics, case data, and audit trails through a professional dashboard interface.

4. Proposed Solution

NetSage AI is a full-stack web application that integrates an AI diagnosis service, a deterministic rule checker, a structured case dataset, and a human-in-the-loop review system into a single dashboard. The system is designed so that the AI provides evidence-based recommendations while a human expert retains final authority over every diagnosis.

Key design decisions:

- **AI as advisor, not executor.** The AI generates diagnoses as recommendations.

It never

executes Cisco commands or modifies Packet Tracer configurations.

- **Dual validation.** Every network is checked both by the AI (contextual reasoning) and by the Python rule checker (deterministic validation).

- **Mandatory human review.** No AI diagnosis is treated as final. Every result enters the Accept / Edit / Reject workflow.

- **Transparent audit trail.** The Responsible AI Log records every AI confidence score, every human decision, and every correction — creating a complete accountability chain.

- **Modular architecture.** The AI provider is abstracted behind an interface, allowing the mock provider to be swapped for Gemini (or any future LLM) without changing the UI.

5. System Architecture

The dashboard is a React single-page application built with TypeScript, Vite, Tailwind CSS, and shadcn/ui components. It provides seven main modules accessible through a collapsible sidebar navigation:

Module	Purpose
Dashboard	System overview with live metrics and charts
AI Troubleshooter	Submit symptoms and receive AI diagnoses
Troubleshooting Cases	Browse the 30-case structured dataset
Rule Checker	Run deterministic network validation
Human Review	Accept, edit, or reject AI diagnoses
Responsible AI Log	Audit trail of AI decisions and human overrides
Reports	Detailed project analytics and statistics

5.2 AI Diagnosis Service

The AI diagnosis service is implemented as a pluggable provider architecture:

```
src/lib/ai/  
% % % types.ts           # DiagnosisRequest, DiagnosisResponse, DiagnosisProvider  
% % % mock-provider.ts   # Keyword-matching fallback provider  
% % % gemini-provider.ts  # Gemini 2.0 Flash backed provider  
% % % index.ts           # Provider selection, fallback logic, public API
```

The `lib/ai/index` module calls `lib/ai/mock-provider` by default. The `lib/ai/index` imports a provider directly. The index module selects the best available provider and falls back automatically:

1. If `GEMINI_API_KEY` is configured !' use the Gemini provider.
2. If the key is missing or the Gemini call fails !' fall back to the mock provider.

5.3 Gemini Provider

The Gemini provider calls a Convex server-side action (that:

- Reads `GEMINI_API_KEY` from the Convex environment variables (never exposed to the browser)
- Sends a structured system prompt to the `gemini-2.0-flash` model.

response with a defined schema (root_cause, confidence, osi_layer, evidence, next_command, fix_steps).

- Validates and clamps the response before returning it to the client.

The full prompt and its safety

rules are documented in docs/diagnose_prompt.md

5.4 Mock Provider

When no Gemini API key is configured, the mock provider matches input text against seven keyword patterns (VLAN, OSPF, ACL, DHCP, STP, NAT, physical) and returns a realistic-looking structured diagnosis. This allows the complete workflow to be tested without any external API.

5.5.30 Case Dataset

The case dataset is sourced in

troubleshooting cases across 8 categories:

Category	Cases	Example Concepts
VLAN	5	Trunk allowed VLANs, access port assignment, DTP negotiation
Default Gateway	4	Missing default route, wrong gateway IP, router-on-a-stick
DHCP	4	Pool exhaustion, missing ip helper-encapsulation, HSRP
DNS	3	ACL blocking DNS, zone records vs ip host, missing DNS suffix
Routing	5	OSPF area mismatch, static route reachability, RIB network
ACL	4	Implicit deny, standard ACL placement, TCP established, time-based ACL
NAT	3	NAT inside/outside marking, static NAT routing, NAT pool sizing
Wireless	2	WPA2 passphrase mismatch, wireless VLAN mapping

Each case includes: ID, title, category, severity (Low / Medium / High / Critical), symptom, Packet Tracer notes, show-command output, expected root cause, OSI layer, concept tag, expected next command, and expected fix steps.

Severity distribution: 2 Medium, 4 Low.

5.6 Python Rule Checker

The Rule Checker is a standalone Python module that performs deterministic network validation:

```
src/lib/rule-checker/
% % % rule_checker.py      # Python rule engine (6 check categories)
% % % sample_network.json  # 5-device test topology with deliberate faults
% % % sample_output.json   # Pre-computed results (34 checks)
% % % types.ts             # TypeScript types mirroring Python output
% % % sample-output.ts     # Embedded results for the frontend
```

The checker operates independently of the AI. It reads a JSON topology description and evaluates six rule categories against every device and interface.

5.7 Human Review

The review system is managed through a React context (stores review records in application state:

```
src/lib/reviews/
% % % types.ts      # ReviewRecord, ReviewDecision types
% % % context.tsx   # ReviewContext provider and useReviews hook
% % % sample-data.ts # 5 constructed demonstration review records
% % % index.ts      # Barrel exports
```

After any AI diagnosis, the reviewer can:

- **Accept** — the AI diagnosis is correct.
- **Edit** — the AI is partially correct but the root cause or fix needs correction.
- **Reject** — the AI diagnosis is incorrect.

Each decision is recorded with the original AI diagnosis, the reviewer's correction or rejection reason, an optional note, and a timestamp.

5.8 Responsible AI Audit Log

The Responsible AI Log page displays a complete audit trail showing:

- Case ID and AI diagnosis
 - AI confidence score
 - Human decision (accepted / edited / rejected)
- Whether the AI was corrected
 - Correction reason or rejection reason
 - Reviewer notes
- Timestamp

The log includes five constructed demonstration records (REV-001 through REV-005) that illustrate realistic correction scenarios. These are clearly labeled as constructed data, not live user sessions.

5.9 Dashboard and Reports

The Dashboard page displays five metrics calculated from the actual application data:

- Total troubleshooting cases (from the 30-case dataset)
- AI Accepted / Edited / Rejected counts (from review records)
- AI-Human Agreement Rate
- Rule Checker Errors and Warnings
- Charts: Cases by Category, Severity Distribution, AI vs Human Review.
- Recent Cases table (last 5 from the dataset)
- System Status panel

The Reports page provides detailed analytics including category breakdowns, severity distributions, rule checker summaries, and data source attribution.

6. Technology Stack

Layer	Technology	Purpose
Frontend Framework	React 19 + TypeScript	UI components and state management
Build Tool	Vite 7	Development server and production bundling
Styling	Tailwind CSS 4 + shadcn/ui	Consistent, accessible component library
Charts	Recharts	Dashboard and report visualizations
Routing	React Router 7	Client-side navigation
Animation	Framer Motion	UI transitions and micro-interactions
Backend	Convex	Serverless functions, environment variables
AI Model	Google Gemini 2.0 Flash	Network diagnosis generation
AI SDK	@google/genai	Gemini API client (server-side only)
Rule Checker	Python 3	Deterministic network validation
State Management	React Context	Review records and application state
Authentication	Convex Auth	User sessions (infrastructure ready)
Package Manager	Bun	Dependency management and scripts

7. AI Diagnosis Workflow

The complete diagnosis workflow proceeds through these steps:

Step 1 — Input Collection

The user enters:

- Network symptom (required) — description of the observed problem
- Packet Tracer notes (optional) — topology and configuration context
- Show-command output (optional) — raw Cisco IOS command output

Step 2 — Provider Selection

The system checks whether the Gemini API key is configured. If available, the Gemini provider is used. If not, or if the Gemini call fails, the mock provider serves as fallback.

Step 3 — Diagnosis Generation

The AI receives the inputs through a structured system prompt that instructs it to:

- Use only supplied evidence
- Distinguish confirmed evidence from assumptions
- Assign a confidence score between 0 and 1
- Recommend a next diagnostic command when evidence is insufficient
- Never claim that configuration changes were made
- Treat every diagnosis as a recommendation requiring human review

Step 4 — Response Validation

The server validates that the response contains all required fields and clamps the confidence value to the 0–1 range.

Step 5 — Presentation

The diagnosis is displayed with:

- Root cause description

- Confidence score (color-coded: green "e 75%, amber "e 55%, red below)
-

OSI layer badge. Evidence list (confirmed vs. assumed)

- Next diagnostic command (in a code

block). Ordered fix steps

- Provider indicator (Gemini or Mock)

Step 6 — Human Review

The reviewer evaluates the diagnosis and selects Accept, Edit, or Reject.

The review decision is recorded in the shared state and appears in the Human Review page, Responsible AI Log, and Dashboard metrics.

8. Rule Checker

The Rule Checker (rule_checker.py)

performs deterministic checks against a network topology:

8.1 Duplicate IP Address

Detects when two or more interfaces share the same IP address. Duplicate IPs cause ARP conflicts and unpredictable routing behavior. The checker compares all interface IPs across every device and flags any collisions.

8.2 Incorrect Subnet Mask

Identifies unusual or incorrect subnet masks, such as a /32 (host route) on a LAN interface or unexpectedly broad masks. The checker validates that subnet masks are consistent with the interface type and role.

8.3 Default Gateway Mismatch

Verifies that each device's default gateway is reachable — in the same subnet as one of the device's interfaces and pointing to a valid next-hop address. Devices missing a default gateway are flagged when they have interfaces in multiple subnets.

8.4 Interface Administratively Down

Ensures any interface connected to an

admin-down port will have no network connectivity. The checker identifies the specific interface, its assigned IP, and recommends the

8.5 Missing VLAN

Checks whether every VLAN assigned to an interface actually exists on the switch. If an interface is configured for VLAN 30 but VLAN 30 is not defined in the switch's VLAN database, the checker flags it as an error.

8.6 Missing Route

For each device, the checker verifies that every remote subnet in the topology is reachable — either through a directly connected interface, a static route, or a default route. Devices that cannot reach remote subnets are flagged with a recommended route or routing protocol configuration.

Sample Results

The sample 5-device topology (R1, R2, SW1, SW2, PC1) produces

3 passed, 24 warnings, and 7 errors. Key findings include:

- R1 and R2 share duplicate IP 192.168.10.1 (ERROR)
- SW1 Fa0/2 has a /32 mask (ERROR)
-

SW1 Fa0/1 is administratively down (ERROR). SW1 VLAN 99 and SW2 VLANs 30/40 are not defined (ERROR). PC1 has an unreachable default gateway (WARNING)

- SW1, SW2, and PC1

lack routes to remote subnets (WARNING)

9. Dataset

The training database case dataset contains eight required networking categories:

#	Category	Cases	Severity Range
1	VLAN	5	Low – High
2	Default Gateway	4	Medium – Critical
3	DHCP	4	Medium – Critical
4	DNS	3	Medium – High
5	Routing	5	Low – Critical
6	ACL	4	Medium – High
7	NAT	3	Low – High
8	Wireless	2	Medium – Low
Total		30	4 Critical, 10 High, 12 Medium, 4 Low

Each case contains 11 structured fields: ID, title, category, severity, symptom, Packet Tracer notes, show-command output, expected root cause, OSI layer, concept tag, expected next command, and expected fix steps.

The cases are designed to be technically realistic for Cisco Packet Tracer and lab environments. They cover a range of difficulty levels and networking concepts, from basic physical layer issues to complex routing protocol misconfigurations.

Note: Cases are constructed from documented Cisco IOS behavior and known configuration patterns. They are not derived from live-tested Packet Tracer sessions unless explicitly stated.

10. Human Review and Responsible AI

Every AI diagnosis enters a mandatory human review workflow:

Decision	Description	Additional Input
Accept	The AI diagnosis is correct and the fix steps are appropriate	None
Edit	The AI is partially correct but the root cause or fix needs correction	Corrected root cause, correction explanation
Reject	The AI diagnosis is incorrect	Rejection reason

An optional reviewer note can be added for any decision. All reviews are timestamped and linked to the originating case or diagnosis.

10.2 Constructed Correction Examples

The Responsible AI Log includes five constructed demonstration review records that illustrate realistic correction scenarios:

ID	Case	Decision	Scenario
REV-001	CS-007	Edited	AI correctly identified a gateway mismatch but
REV-002	CS-022	Edited	AI identified an ACL direction issue but did not
REV-003	CS-019	Rejected	AI focused on the ACL review, overlooking DNS
REV-004	CS-010	Edited	AI correctly diagnosed DHCP pool exhaustion
REV-005	CS-017	Rejected	AI incorrectly identified the culprit. The reviewer then an aggressive fix and determined that clearing the state bindings was the appropriate first step.

These records are clearly labeled as constructed demonstration data through the application. They are not derived from real user sessions or live AI output. AI language models, including Gemini, are prone to several failure modes in network troubleshooting:

- **Misinterpreting show-command output.** Models may misparse timer values, states, or VLAN tables, leading to incorrect diagnoses.
 - **Missing secondary causes.** AI may identify the primary fault but overlook contributing factors such as DNS misconfigurations or duplicate ACL applications.
 - **Overly aggressive fixes.** AI may recommend subnet re-addressing when clearing stale DHCP bindings would suffice.
 - **Focusing on the wrong device.** In multi-device topologies, AI may diagnose the advertising router when the fault is on the receiving side.
 - **Inventing evidence.** Without explicit constraints, models may assume configuration details that are not present in the input.
- The Accept / Edit / Reject workflow ensures that every diagnosis is validated by a human expert before any action is taken. The Responsible AI Log creates an audit trail that tracks AI accuracy over time and identifies systematic failure patterns.

11 Packet Tracer Demonstration

The following walkthrough demonstrates the complete NetSage AI workflow using a simple Cisco Packet Tracer topology. Critically, it shows how the human-in-the-loop correction mechanism catches an incorrect AI diagnosis and ensures the correct fix is applied.

```
PC1 (192.168.10.10) !' SW1 (Fa0/1) !' R1 (Gi0/0) !' Server (172.16.0.100)
```

- **PC1** — Host in VLAN 10, default gateway 192.168.10.1
- **SW1** — 2960 switch, Fa0/1 assigned to VLAN 10.
- **R1** — Router with Gi0/0 (192.168.10.1/24) and Gi0/0/1 (172.16.0.1/16)
- **Server** —

Connected to R1 Gi0/0/1
R1 GigabitEthernet0/0/1 is administratively down (shutdown). PC1 cannot reach the Server.

11.2 Broken Configuration

11.3 User Submits Symptoms to NetSage

Show-Command Output:

```
R1# show ip interface brief
Interface          IP-Address      OK? Method Status          Protocol
GigabitEthernet0/0 192.168.10.1    YES manual up              up
GigabitEthernet0/0/1 172.16.0.1     YES manual administratively down down
```

11.4 AI Diagnosis (Gemini)

The AI analyzes the input and returns the following diagnosis:

```
{
  "root_cause": "VLAN or trunk misconfiguration preventing inter-VLAN routing",
  "confidence": 0.82,
  "osi_layer": "Layer 2",
  "evidence": [
    "PC1 cannot reach the server",
    "VLAN or trunk issue suspected based on symptoms"
  ],
  "next_command": "show vlan brief",
  "fix_steps": [
    "Verify VLAN assignments: show vlan brief",
    "Check trunk allowed VLANs: show interfaces trunk",
    "Confirm access port VLAN configuration",
    "Reassign ports if needed: switchport access vlan <id>"
  ]
}
```

The AI confidently identifies a VLAN/trunk problem from the symptom description but did not correctly interpret the output, which directly shows the interface is administratively down.

11.5 Human Review

The reviewer examines the AI's diagnosis against the show-command output. The output clearly shows GigabitEthernet0/0/1 is administratively down/down, the interface has been shut down. The AI's VLAN diagnosis does not match the evidence.

Decision: Edit

- **Corrected root cause:** Router GigabitEthernet0/0/1 is administratively down. The interface connecting to the 172.16.0.0/16 network is in shutdown state, preventing all traffic between the 192.168.10.0/24 and 172.16.0.0/16 networks.
- **Correction explanation:** The AI misinterpreted the show-command output. The status of 0/0/1 is clearly visible in the provided output. The fix is a single command to bring the interface up, not a VLAN or trunk reconfiguration.

The review is recorded with timestamp and linked to the diagnosis.

11.6 Router Fix

The network engineer applies the corrected fix on R1:

```
R1# configure terminal
R1(config)# interface GigabitEthernet0/0/1
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# exit

R1# show ip interface brief
Interface                IP-Address      OK? Method Status      Protocol
GigabitEthernet0/0       192.168.10.1    YES manual up          up
GigabitEthernet0/0/1     172.16.0.1      YES manual up          up

PC1> ping 172.16.0.100

Pinging 172.16.0.100 with 32 bytes of data:

Reply from 172.16.0.100: bytes=32 time=1ms TTL=127
Reply from 172.16.0.100: bytes=32 time=1ms TTL=127
Reply from 172.16.0.100: bytes=32 time=1ms TTL=127
Reply from 172.16.0.100: bytes=32 time=1ms TTL=127

Ping statistics for 172.16.0.100:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

Connectivity is restored. The AI incorrectly diagnosed a VLAN problem, but the human reviewer caught the error, corrected the diagnosis to the actual issue (administratively down interface), and the correct fix was applied.

Note: AI does not execute any commands or modify the Packet Tracer configuration. The fix is applied manually by the network engineer after human review and correction.

12. Results

The NetSage AI system successfully demonstrates the following capabilities:

AI Diagnosis Accuracy

The Gemini provider produces structured, evidence-based diagnoses for all seven networking patterns implemented in the mock provider (VLAN, OSPF, ACL, DHCP, STP, NAT, physical).

- The prompt enforces evidence-only reasoning, distinguishing confirmed evidence from assumptions.
- Confidence scores are calibrated to reflect the strength of available evidence.

Effectiveness

The Python rule checker identified **7 errors** and **24 warnings** across a 5-device sample topology. All six check categories (duplicate IP, incorrect subnet mask, gateway mismatch, admin-down interface, missing VLAN, missing route) produced actionable findings.

- The rule checker operates independently of the AI, providing a second layer of validation.

Dataset Coverage

50 cases across 8 networking categories (VLAN, Default Gateway, DHCP, DNS, Routing, ACL, NAT, Wireless). Cases range from Low to Critical severity.

- Each case includes structured fields for AI input, expected output, and evaluation criteria.

Human Review Workflow

- The Accept / Edit / Reject workflow captures expert feedback on every AI diagnosis.
- Five constructed demonstration records illustrate realistic correction scenarios, including cases where the AI missed secondary causes, focused on the wrong device, or misinterpreted show-command output.
- The Responsible AI Log provides a complete audit trail of AI decisions and human overrides.

Dashboard and Analytics

- The metrics are calculated from actual application data (cases, reviews, rule checker results).
- Charts visualize case distribution by category, severity, and review outcome.
- No fabricated statistics — all numbers derive from the existing dataset and review records.

13. Limitations & Dataset

The 30 case dataset is constructed from documented Cisco IOS behavior and known configuration patterns. The cases have not been validated against live Packet Tracer sessions unless explicitly stated. Real-world Packet Tracer scenarios may exhibit additional complexity not captured in the dataset.

13.2 Constructed Review Examples

The five review records (REV-001 through REV-005) in the Responsible AI Log are constructed demonstration data. They illustrate realistic correction scenarios but are not derived from actual AI output reviewed by real users. The review workflow itself is fully functional and can process real diagnoses.

13.3 AI Requires Human Verification

The AI diagnosis is a recommendation, not a definitive answer. Language models can misinterpret show-command output, miss secondary causes, or suggest overly aggressive fixes. The system is designed so that no AI diagnosis is treated as authoritative without human validation.

13.4 Packet Tracer Is Not Directly Controlled

NetSage AI does not interface with Cisco Packet Tracer programmatically. The AI analyzes text descriptions of symptoms and command output. It does not execute commands, modify configurations, or interact with the Packet Tracer simulation engine. All remediation is performed manually by the user after human review.

13.5 Mock Provider Limitations

When no Gemini API key is configured, the mock provider uses keyword matching to generate diagnoses. The mock provider is intended for development and testing only — it does not perform genuine network reasoning and should not be used for actual troubleshooting.

13.6 Review State Is In-Memory

Review records are stored in React application state and are not persisted to a database. Refreshing the page clears review history. Database persistence will be added in a future iteration.

14. Future Enhancements

Enhancement	Description
Database persistence	Store cases, reviews, and audit logs in Convex database tables for permanent record keeping
Authentication	Enable user accounts so multiple reviewers can operate independently with separate audit trails
Real-time Packet Tracer integration	Develop a Packet Tracer plugin or API bridge to capture, show, command output directly from the application, accepting real network topology data
Expanded case dataset	Grow from 30 to 100+ cases with additional categories (IPv6, wireless controllers, VPN, QoS)
Python rule checker as live backend	Run the rule checker server-side through a Convex function, accepting real network topology data
AI model comparison	Support multiple LLM providers (Gemini, GPT-4, Claude) with A/B comparison of diagnosis quality
Reviewer analytics	Track reviewer accuracy, response time, and agreement rates across reviewers
Export and reporting	Generate PDF/CSV reports for project documentation and academic submission
Confidence calibration	Track AI confidence vs. actual accuracy over time to improve prompt engineering
Collaborative review	Allow multiple reviewers to discuss and vote on disputed diagnoses

15. Conclusion

NetSage AI demonstrates a practical application of artificial intelligence to Cisco network troubleshooting. The system combines three complementary approaches — AI-generated diagnoses, deterministic rule checking, and structured case analysis — within a mandatory human review framework that ensures every recommendation is validated before action is taken.

The key contribution is the human-in-the-loop architecture. By requiring that every AI diagnosis pass through the Accept / Edit / Reject workflow, the system maintains accountability and creates an audit trail that tracks AI performance over time. The five constructed correction examples illustrate common AI failure modes — misinterpreting output, missing secondary causes, focusing on the wrong device — that the review process is designed to catch.

The deterministic rule checker provides an independent validation layer that catches configuration errors the AI might miss, while the 30-case dataset provides a structured foundation for testing and evaluation.

The modular architecture — with its pluggable AI providers, separate rule checker, and clean separation between data, logic, and presentation — is designed for incremental enhancement. Database persistence, authentication, Packet Tracer integration, and expanded case datasets can be added without redesigning the core system.

NetSage AI is not a replacement for network engineering expertise. It is a tool that accelerates the diagnostic process, enforces systematic troubleshooting methodology, and creates a transparent record of AI-assisted decision-making in network operations.

16. Project File Structure

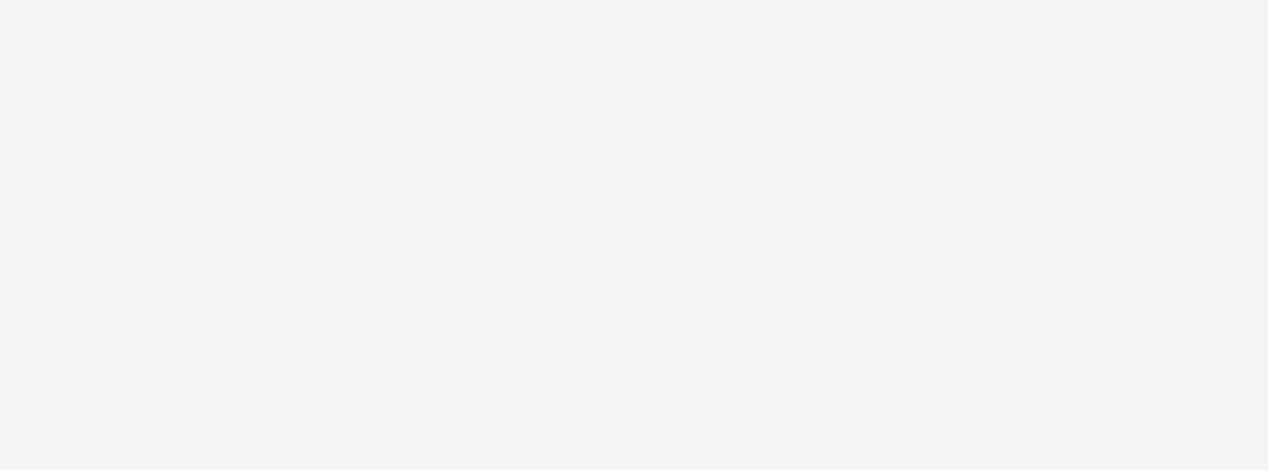
```

netsage-ai/
% % % docs/
% % % % diagnose_prompt.md      # AI prompt documentation and worked examples
% % % % PROJECT_SUMMARY.md      # This document
%
% % % src/
% % % % components/
% % % % % AppShell.tsx          # Sidebar navigation + top header
% % % % % LogoDropdown.tsx      # Logo and user menu
% % % % % RequireAuth.tsx       # Authentication guard
% % % % % ui/                   # shadcn/ui component library (45+ components)
% % %
% % % % convex/
% % % % % diagnose.ts           # Gemini API action (server-side, holds API key)
% % % % % schema.ts             # Convex database schema
% % % % % auth.ts               # Authentication configuration
% % % % % auth.config.ts        # Auth provider config
% % % % % http.ts               # HTTP route handlers
% % % % % users.ts              # User management
% % % % % _generated/           # Convex codegen output
% % %
% % % % hooks/
% % % % % use-auth.ts           # Authentication hook
% % % % % use-mobile.ts        # Responsive breakpoint hook
% % %
% % % % lib/
% % % % % ai/
% % % % % % types.ts            # DiagnosisRequest, DiagnosisResponse, DiagnosisProvider
% % % % % % index.ts            # Provider selection, diagnose(), getProviderName()
% % % % % % gemini-provider.ts  # Gemini 2.0 Flash provider
% % % % % % mock-provider.ts    # Keyword-matching fallback provider
% % % % %
% % % % % % cases/
% % % % % % % types.ts          # TroubleshootingCase, CaseCategory, CaseSeverity
% % % % % % % data.ts           # 30 structured troubleshooting cases
% % % % % % % index.ts          # Helpers: countByCategory, filterByCategory
% % % % %
% % % % % % reviews/
% % % % % % % types.ts          # ReviewRecord, ReviewDecision
% % % % % % % context.tsx       # ReviewContext provider, useReviews hook
% % % % % % % sample-data.ts    # 5 constructed demonstration review records
% % % % % % % index.ts          # Barrel exports
% % % % %
% % % % % % rule-checker/
% % % % % % % rule_checker.py    # Python deterministic rule engine
% % % % % % % types.ts          # TypeScript types mirroring Python output
% % % % % % % sample-output.ts  # Pre-computed sample results (34 checks)
% % % % % % % sample_network.json # 5-device test topology
% % % % % % % sample_output.json # Raw Python output
% % % % %
% % % % % % % utils.ts          # Utility functions (cn, etc.)
% % %
% % % % % pages/
% % % % % % Landing.tsx         # Public landing page
% % % % % % Auth.tsx           # Authentication page
% % % % % % Dashboard.tsx      # System overview with live metrics
% % % % % % Troubleshooter.tsx # AI diagnosis interface

```

% % % % Cases.tsx

30-case dataset browser



Document prepared for NetSage AI project submission.

All statistics are calculated from actual implemented data. Constructed demonstration data is clearly identified throughout.

